1. Introduction to Python Libraries- Numpy, Pandas, Matplotlib, Scikit

```
[ ]
import numpy as np
a = np.array([1,2,3,4,5])
b= np.arange(1,10,2)
c=np.linspace(0,1,5)
d=np.zeros((2,3))
e=np.ones((2,3))
f=np.eye(3)
g=np.random.randint(1,100,size=(3,3))

print("array a:\n",a)
print("arange:\n",b)
print("linspace:\n",c)
print("zeros:\n",d)
print("ones:",e)
print("eye:\n",f)
print("random:\n",g)
```

output:

```
array a:
 [1 2 3 4 5]
arange:
 [1 3 5 7 9]
linspace:
 [0.   0.25 0.5  0.75 1.  ]
zeros:
 [[0. 0. 0.]
 [0. 0. 0.]]
ones: [[1. 1. 1.]
 [1. 1. 1.]]
eye:
 [[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
random:
 [[67 74 28]
 [60 49 34]
 [22 28 40]]
```

```python
print("\nshape:",g.shape)
print("dimension:",g.ndim)
print("size:",g.size)
print("data type",g.dtype)
```

```
shape: (3, 3)
dimension: 2
size: 9
data type int64
```

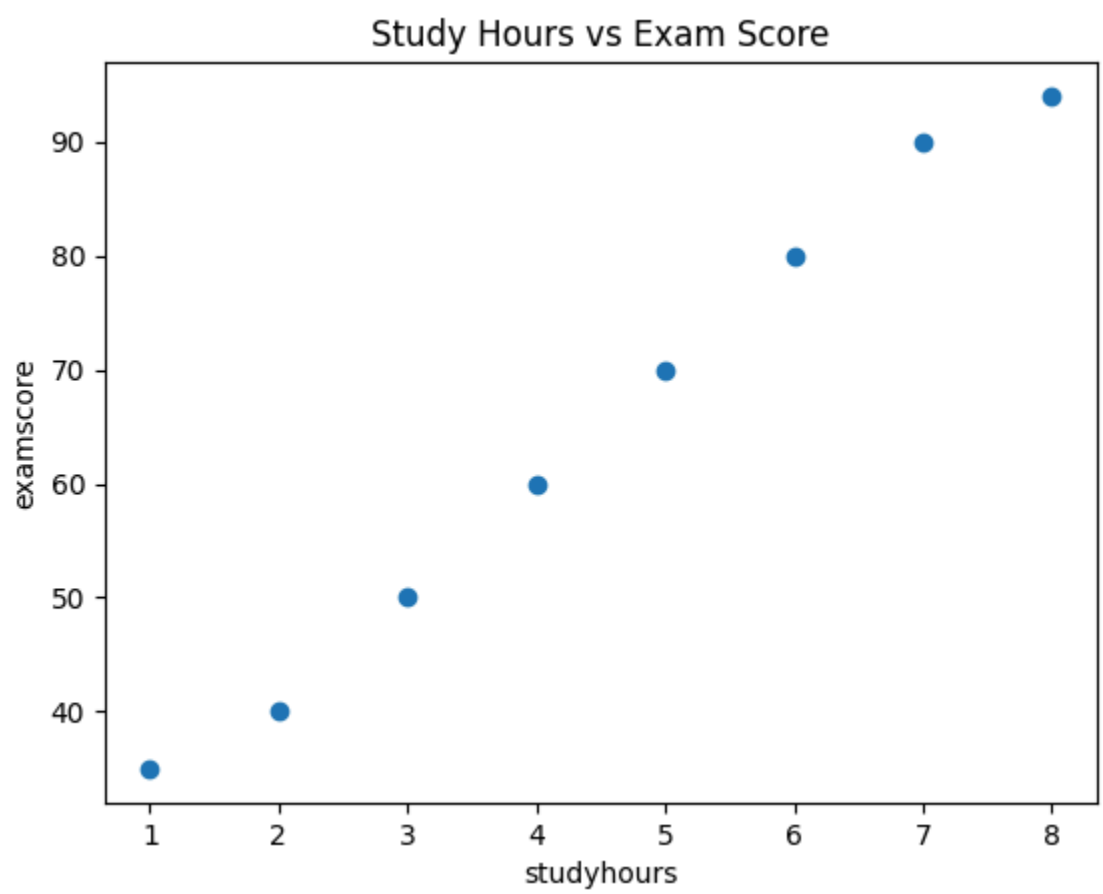
```python
reshaped=g.reshape(1,9)
print("\n reshape array:",reshaped)
print("transpose:\n",g.T)
print("flattened:\n",g.flatten())
```

```
 reshape array: [[67 74 28 60 49 34 22 28 40]]
transpose:
 [[67 60 22]
 [74 49 28]
 [28 34 40]]
flattened:
 [67 74 28 60 49 34 22 28 40]
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np
data={
    'studyhours':[1,2,3,4,5,6,7,8],
    'examscore':[35,40,50,60,70,80,90,94]

}
df=pd.DataFrame(data)
```

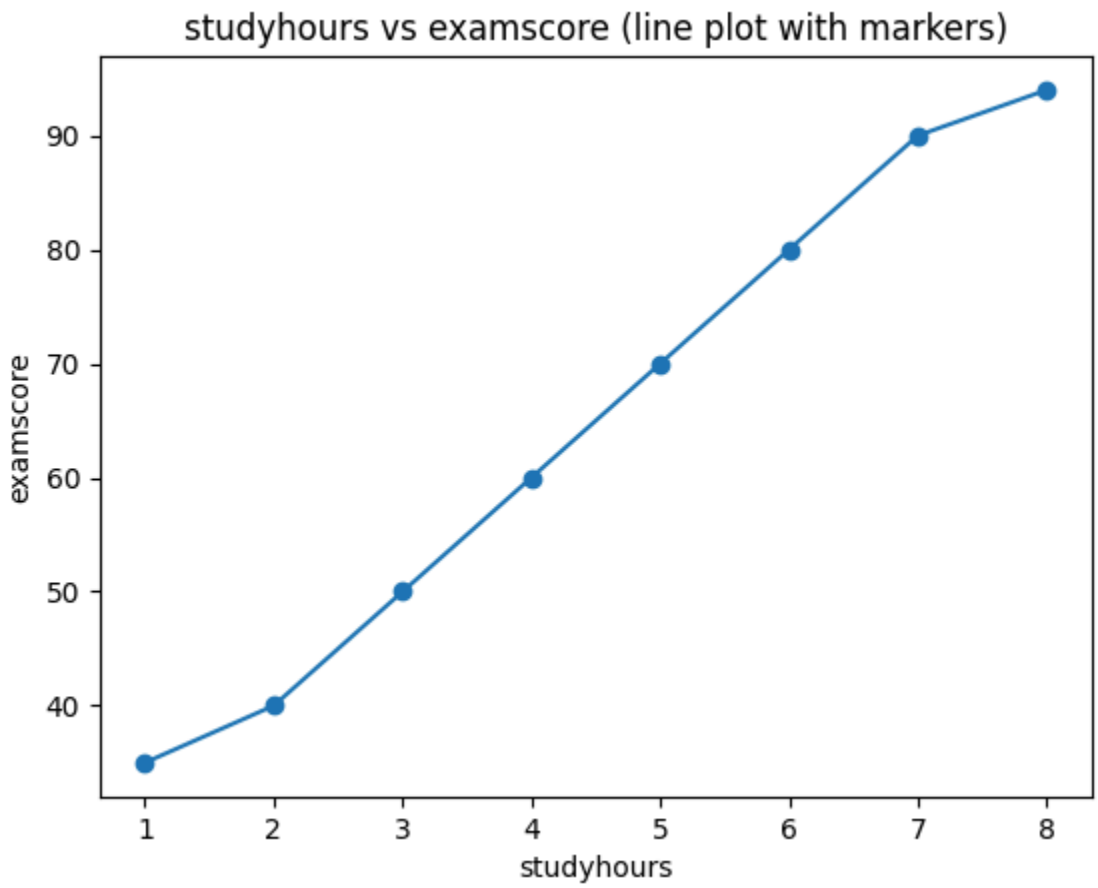
```python
plt.scatter(df['studyhours'],df['examscore'])
plt.xlabel("studyhours")
plt.ylabel("examscore")
plt.title("Study Hours vs Exam Score")
plt.show()
```

output:



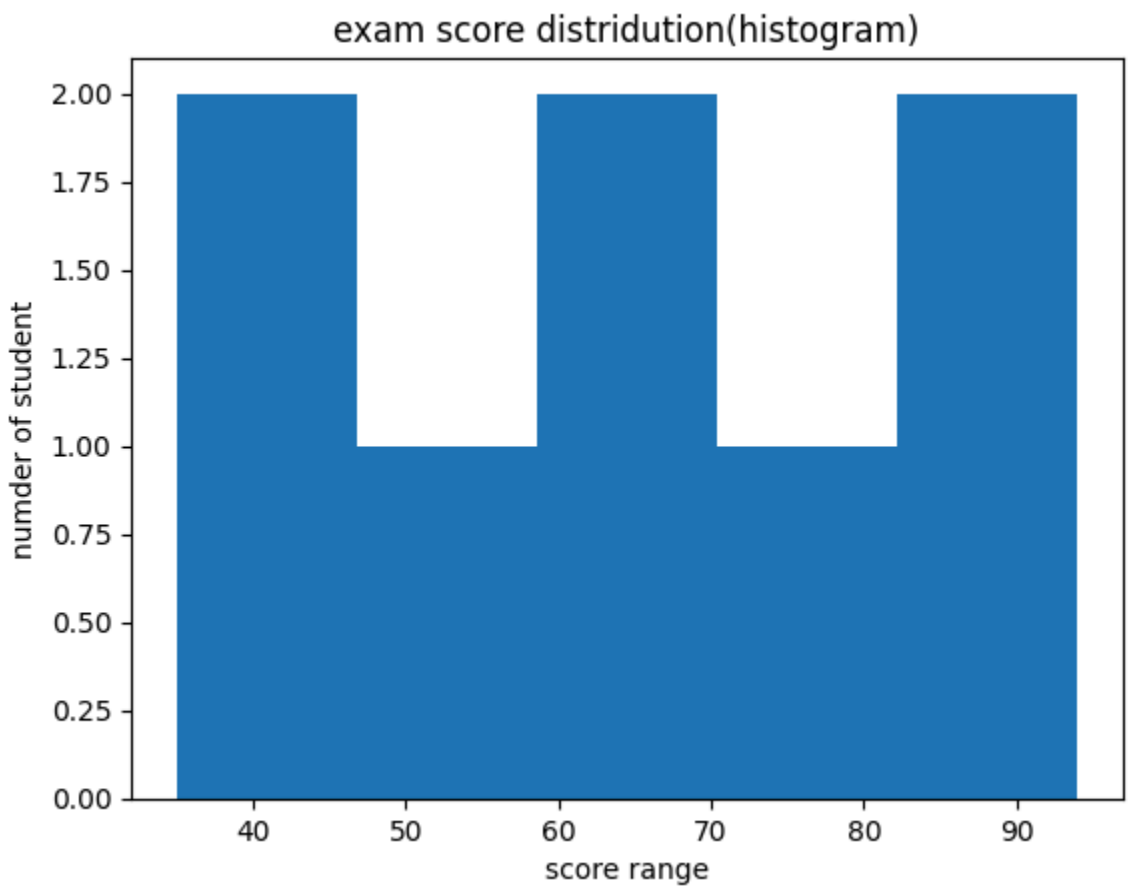
```
plt.plot(df['studyhours'],df['examscore'],marker='o')
plt.xlabel("studyhours")
plt.ylabel("examscore")
plt.title("studyhours vs examscore (line plot with markers)")
plt.show()
```

output:



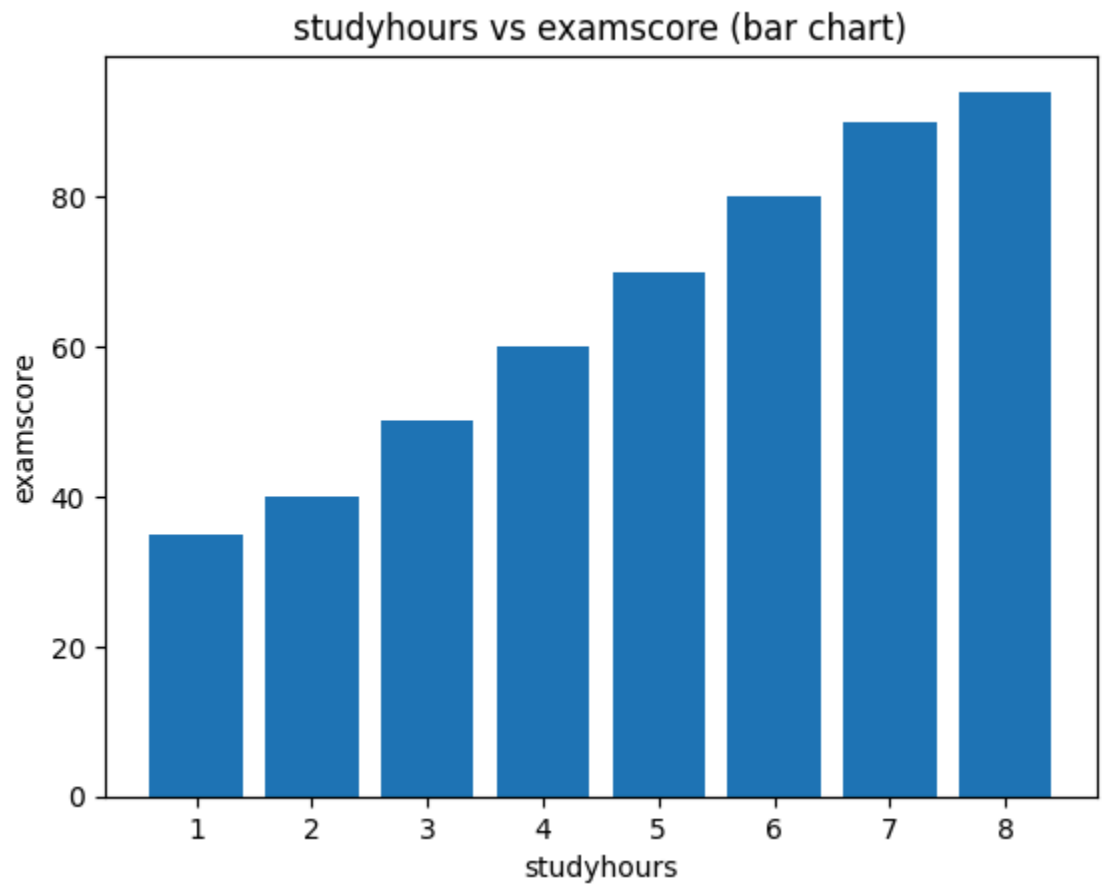
```
plt.hist(df['examscore'],bins=5)
plt.xlabel("score range")
plt.ylabel("numder of student")
plt.title("exam score distridution(histogram)")
plt.show()
```

output:



```
plt.bar(df['studyhours'],df['examscore'])
plt.xlabel("studyhours")
plt.ylabel("examscore")
plt.title("studyhours vs examscore (bar chart)")
plt.show()
```

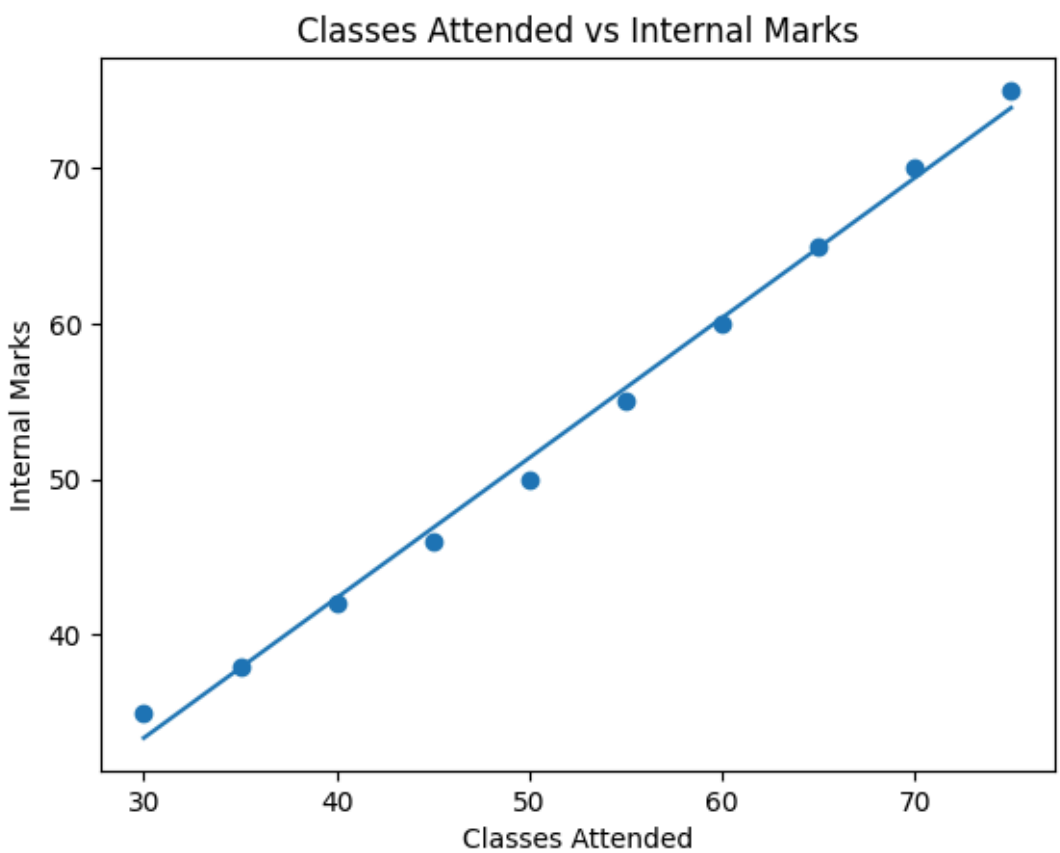
output:



```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error
data = {
'Student': ['S1','S2','S3','S4','S5','S6','S7','S8','S9','S10'],
'Classes_Attended': [30, 35, 40, 45, 50, 55, 60, 65, 70, 75],
'Internal_Marks': [35, 38, 42, 46, 50, 55, 60, 65, 70, 75]
}
df = pd.DataFrame(data)
print(df)
X = df[['Classes_Attended']]
y = df['Internal_Marks']
X_train, X_test, y_train, y_test = train_test_split(
X, y, test_size=0.2, random_state=7
)
model = LinearRegression()
model.fit(X_train, y_train)
predictions = model.predict(X_test)
mse = mean_squared_error(y_test, predictions)
print("MSE:", mse)
print("Marks per Class:", model.coef_[0])
print("Base Marks:", model.intercept_)
plt.scatter(X, y)
plt.plot(X, model.predict(X))
plt.xlabel("Classes Attended")
plt.ylabel("Internal Marks")
plt.title("Classes Attended vs Internal Marks")
plt.show()
```

Output:

```
...    Student  Classes_Attended  Internal_Marks
    0      S1                30              35
    1      S2                35              38
    2      S3                40              42
    3      S4                45              46
    4      S5                50              50
    5      S6                55              55
    6      S7                60              60
    7      S8                65              65
    8      S9                70              70
    9     S10                75              75
    MSE: 0.578125
    Marks per Class: 0.9
    Base Marks: 6.375
```



2.Perform Data exploration and preprocessing in Python

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
data={
    'country':['india','usa','india','uas','uk','india'],
    'age':[22,25,np.nan,30,28,35],
    'salary':['400000','600000','50000',np.nan,'234000','600000'],
    'purchased':['no','yes','yes','no','yes','yes']
}
df=pd.DataFrame(data)
print("--original rawdata--")
print(df)
print("\n")
print("--missing values count--")
print(df.isnull().sum())
print("\n")
df['age']=df['age'].fillna(df['age'].mean())
df['salary']=df['salary'].fillna(df['salary'].astype(float).mean())
print("--data after cleaning--")
print(df)
print("\n")
df_encoded=pd.get_dummies(df,columns=['country'])
df_encoded['purchased']=df_encoded['purchased'].map({'yes':1,'no':0})
print("--data after encoding (all numbers now)--")
print(df_encoded)
print("\n")
x=df_encoded.drop('purchased',axis=1)
y=df_encoded['purchased']
scale=StandardScaler()
x_scaled=scale.fit_transform(x)
print("--final processed data--")
print(pd.DataFrame(x_scaled,columns=x.columns).head())
```

Output:

```
--original rawdata--
  country   age  salary purchased
0   india  22.0  400000        no
1     usa  25.0  600000       yes
2   india   NaN   50000       yes
3     uas  30.0     NaN        no
4      uk  28.0  234000       yes
5   india  35.0  600000       yes


--missing values count--
country      0
age          1
salary       1
purchased    0
dtype: int64


--data after cleaning--
  country   age    salary purchased
0   india  22.0    400000        no
1     usa  25.0    600000       yes
2   india  28.0     50000       yes
3     uas  30.0  376800.0        no
4      uk  28.0    234000       yes
5   india  35.0    600000       yes
```

```
--data after encoding (all numbers now)--
    age    salary  purchased  country_india  country_uas  country_uk  \
0  22.0    400000          0           True        False       False
1  25.0    600000          1          False        False       False
2  28.0     50000          1           True        False       False
3  30.0  376800.0          0          False         True       False
4  28.0    234000          1          False        False        True
5  35.0    600000          1           True        False       False

   country_usa
0        False
1         True
2        False
3        False
4        False
5        False


--final processed data--
        age    salary  country_india  country_uas  country_uk  country_usa
0 -1.484615  0.119180            1.0    -0.447214   -0.447214    -0.447214
1 -0.742307  1.146590           -1.0    -0.447214   -0.447214     2.236068
2  0.000000 -1.678789            1.0    -0.447214   -0.447214    -0.447214
3  0.494872  0.000000           -1.0     2.236068   -0.447214    -0.447214
4  0.000000 -0.733571           -1.0    -0.447214    2.236068    -0.447214
```

3. Implement regularised Linear regression

```python
from sklearn.linear_model import LinearRegression,Ridge,Lasso
from sklearn.metrics import mean_squared_error


diabetes = datasets.load_diabetes()
x_train, x_test, y_train, y_test = train_test_split(X_diabetes, y_diabetes, test_size=0.2, random_state=42)

lr = LinearRegression().fit(x_train, y_train)
ridge = Ridge(alpha=1.0).fit(x_train, y_train)
lasso = Lasso(alpha=1.0).fit(x_train, y_train)

print(f"Linear Regression MSE: {mean_squared_error(y_test, lr.predict(x_test)):.2f}")
print(f"Ridge Regression MSE: {mean_squared_error(y_test, ridge.predict(x_test)):.2f}")
print(f"Lasso Regression MSE: {mean_squared_error(y_test, lasso.predict(x_test)):.2f}")
```

Output:

```
Linear Regression MSE: 2900.19
Ridge Regression MSE: 3077.42
Lasso Regression MSE: 3403.58
```

4. Implement Naive Bayes classifier for dataset stored as CSV file.

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.naive_bayes import GaussianNB
from sklearn.metrics import accuracy_score
data={
    'studyhours':[5,2,8,1,6,3,7,4],
    'attendance':[80,50,90,40,85,60,88,65],
    'Result':['pass','fail','pass','fail','pass','fail','pass','fail']
}
df=pd.DataFrame(data)
x=df[['studyhours','attendance']]
y=df['Result']
x_train,x_test,y_train,y_test=train_test_split(x,y,test_size=0.2,random_state=42)
model=GaussianNB()
model.fit(x_train,y_train)
y_pred=model.predict(x_test)
accuracy=accuracy_score(y_test,y_pred)
print("predicted result:",y_pred)
print("actutal result", y_test.values)
print("accuracy:",accuracy)
```

Output:

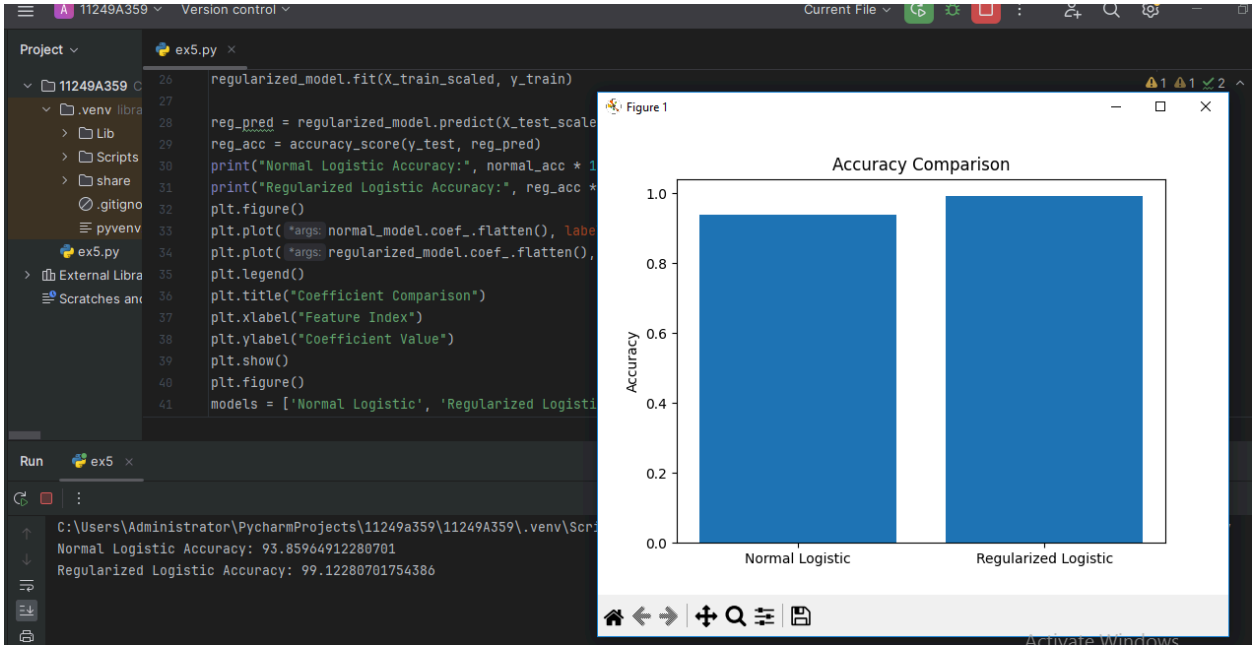
```
predicted result: ['fail' 'fail']
actutal result ['fail' 'fail']
accuracy: 1.0
```

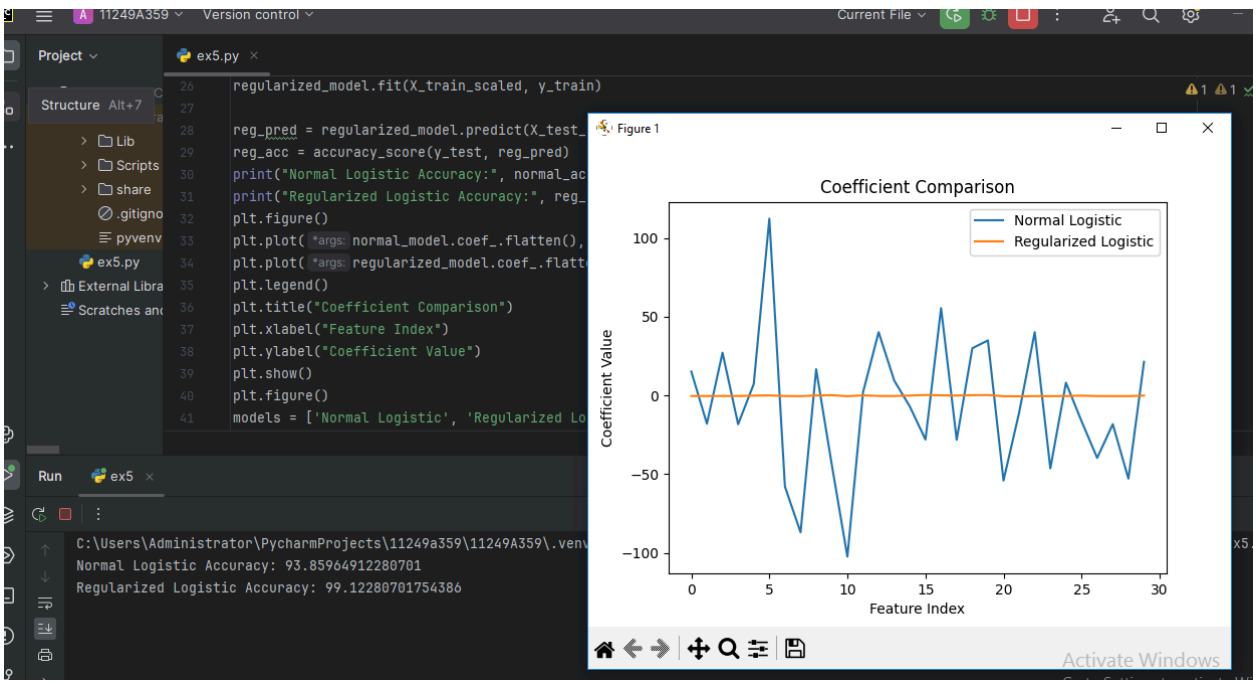
5. Implement regularized logistic regression

```python
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_breast_cancer

data = load_breast_cancer()
X = data.data
y = data.target
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: X, y, test_size=0.2, random_state=42
)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
normal_model = LogisticRegression(C=1e6, penalty='l2', solver='liblinear', max_iter=1000)
normal_model.fit(X_train_scaled, y_train)

normal_pred = normal_model.predict(X_test_scaled)
normal_acc = accuracy_score(y_test, normal_pred)
regularized_model = LogisticRegression(C=0.1, penalty='l2', solver='liblinear', max_iter=1000)
regularized_model.fit(X_train_scaled, y_train)
```

```python
regularized_model.fit(X_train_scaled, y_train)

reg_pred = regularized_model.predict(X_test_scaled)
reg_acc = accuracy_score(y_test, reg_pred)
print("Normal Logistic Accuracy:", normal_acc * 100)
print("Regularized Logistic Accuracy:", reg_acc * 100)
plt.figure()
plt.plot( *args: normal_model.coef_.flatten(), label='Normal Logistic')
plt.plot( *args: regularized_model.coef_.flatten(), label='Regularized Logistic')
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
plt.figure()
models = ['Normal Logistic', 'Regularized Logistic']
accuracies = [normal_acc, reg_acc]

plt.bar(models, accuracies)
plt.title("Accuracy Comparison")
plt.ylabel("Accuracy")
plt.show()
```

Output:


regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_scale
reg_acc = accuracy_score(y_test, reg_pred)
print("Normal Logistic Accuracy:", normal_acc * 1
print("Regularized Logistic Accuracy:", reg_acc *
plt.figure()
plt.plot(*args: normal_model.coef_.flatten(), labe
plt.plot(*args: regularized_model.coef_.flatten(),
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
plt.figure()
models = ['Normal Logistic', 'Regularized Logisti
C:\Users\Administrator\PycharmProjects\11249a359\11249A359\.venv\Scri
Normal Logistic Accuracy: 93.85964912280701
Regularized Logistic Accuracy: 99.12280701754386
Figure 1
Accuracy Comparison
Accuracy
1.0
0.8
0.6
0.4
0.2
0.0
Normal Logistic
Regularized Logistic


regularized_model.fit(X_train_scaled, y_train)
reg_pred = regularized_model.predict(X_test_
reg_acc = accuracy_score(y_test, reg_pred)
print("Normal Logistic Accuracy:", normal_ac
print("Regularized Logistic Accuracy:", reg_
plt.figure()
plt.plot(*args: normal_model.coef_.flatten(),
plt.plot(*args: regularized_model.coef_.flatt
plt.legend()
plt.title("Coefficient Comparison")
plt.xlabel("Feature Index")
plt.ylabel("Coefficient Value")
plt.show()
plt.figure()
models = ['Normal Logistic', 'Regularized Lo
C:\Users\Administrator\PycharmProjects\11249a359\11249A359\.venv
Normal Logistic Accuracy: 93.85964912280701
Regularized Logistic Accuracy: 99.12280701754386
Figure 1
Coefficient Comparison
Normal Logistic
Regularized Logistic
Coefficient Value
100
50
0
−50
−100
0
5
10
15
20
25
30
Feature Index

6. Build models using different Ensembling techniques

```python
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
from sklearn.ensemble import RandomForestClassifier, AdaBoostClassifier, StackingClassifier
from sklearn.svm import SVC
from sklearn.linear_model import LogisticRegression
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: iris.data, iris.target, test_size=0.2, random_state=42)
rf = RandomForestClassifier(n_estimators=100).fit(X_train, y_train)
ada = AdaBoostClassifier(n_estimators=100).fit(X_train, y_train)
estimators = [
    ('rf', RandomForestClassifier()),
    ('svc', SVC(probability=True))
]
stack = StackingClassifier(
    estimators=estimators,
    final_estimator=LogisticRegression())
stack.fit(X_train, y_train)
rf_pred = rf.predict(X_test)
ada_pred = ada.predict(X_test)
stack_pred = stack.predict(X_test)
rf_acc = accuracy_score(y_test, rf_pred)
ada_acc = accuracy_score(y_test, ada_pred)
stack_acc = accuracy_score(y_test, stack_pred)
print(f"Random Forest Accuracy: {rf_acc:.2f}")
print(f"AdaBoost Accuracy: {ada_acc:.2f}")
```

output:

```
C:\Users\Administrator\PycharmProjects\11249a359\11249A359\.venv\Scripts\python.exe C:\Users\Administr
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Process finished with exit code 0
```

7.Build models using Decision trees

```python
from sklearn import datasets
from sklearn.tree import DecisionTreeClassifier
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score
iris = datasets.load_iris()
X_train, X_test, y_train, y_test = train_test_split(
    *arrays: iris.data,
    iris.target,
    test_size=0.2,
    random_state=42)
dt_clf = DecisionTreeClassifier(max_depth=3)
dt_clf.fit(X_train, y_train)
y_pred = dt_clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f"Decision Tree Classifier Accuracy: {accuracy:.2f}")
print("Feature Importances:", dt_clf.feature_importances_)
```

output:

```
C:\Users\Administrator\PycharmProjects\11249a359\11249A359\.venv\Scripts\python.exe C:\Users\Administrator\PycharmProjects\11249a359\11249A359\ex6.py
Random Forest Accuracy: 1.00
AdaBoost Accuracy: 0.93
Stacking Accuracy: 1.00

Process finished with exit code 0
```

8. Build model using SVM with different kernels

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
data = load_iris()
X = data.data
y = data.target   # fixed here (was targett)
X_train, X_test, y_train, y_test = train_test_split(*arrays: X, y, test_size=0.2)
model_linear = SVC(kernel='linear')
model_linear.fit(X_train, y_train)
pred_linear = model_linear.predict(X_test)
acc_linear = accuracy_score(y_test, pred_linear)
print("Linear Kernel Accuracy:", acc_linear)
model_poly = SVC(kernel='poly', degree=3)
model_poly.fit(X_train, y_train)
pred_poly = model_poly.predict(X_test)
acc_poly = accuracy_score(y_test, pred_poly)
print("Polynomial Kernel Accuracy:", acc_poly)
model_rbf = SVC(kernel='rbf')
model_rbf.fit(X_train, y_train)
pred_rbf = model_rbf.predict(X_test)
acc_rbf = accuracy_score(y_test, pred_rbf)
print("RBF Kernel Accuracy:", acc_rbf)
```

Output:

```
C:\Users\Administrator\PycharmProjects\11249a359\11249A359\.venv\Scripts\python.exe C:\Users\Administrator\PycharmProjects\11249a359\11249A359\ex8.py
Linear Kernel Accuracy: 0.9666666666666667
Polynomial Kernel Accuracy: 0.9666666666666667
RBF Kernel Accuracy: 0.9333333333333333

Process finished with exit code 0
```

9.Build model using SVM with different kernels

```python
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score, confusion_matrix, classification_report
from sklearn.datasets import load_iris
iris = load_iris()
data = pd.DataFrame(iris.data, columns=iris.feature_names)
data["price_range"] = iris.target

X = data.drop( labels: "price_range", axis=1)
y = data["price_range"]
X_train, X_test, y_train, y_test = train_test_split(
        *arrays: X, y, test_size=0.2, random_state=42)
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train, y_train)
y_pred = knn.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("Classification Report:\n", classification_report(y_test, y_pred))
```

Output:

```
    accuracy                           1.00        30
   macro avg       1.00      1.00      1.00        30
weighted avg       1.00      1.00      1.00        30

Process finished with exit code 0
```

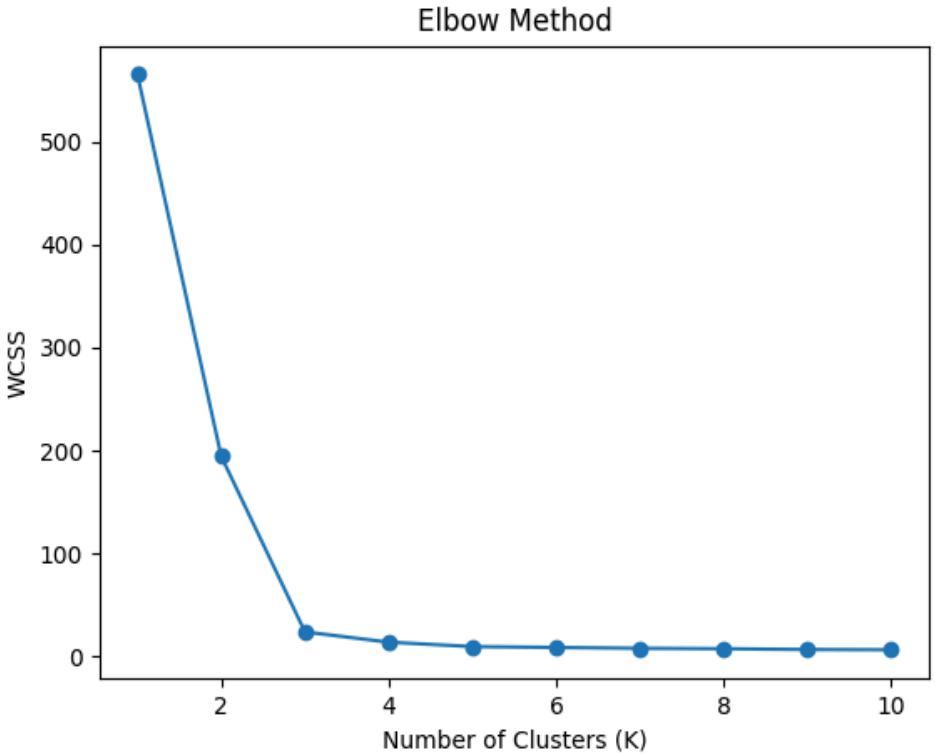
10. Build model to perform Clustering using K-means after applying PCA and determining the value of K using Elbow Method

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA
from sklearn.cluster import KMeans
from sklearn.datasets import make_blobs
X, _ = make_blobs(n_samples=200, centers=5, n_features=3, random_state=42)
data = pd.DataFrame(X, columns=[
    'Age', 'Annual Income (k$)', 'Spending Score (1-100)'])
X = data[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Explained Variance Ratio:", pca.explained_variance_ratio_)
wcss = []
for i in range(1, 11):
    kmeans = KMeans(n_clusters=i, random_state=0)
    kmeans.fit(X_pca)
    wcss.append(kmeans.inertia_)
plt.plot( *args: range(1, 11), wcss, marker='o')
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(X_pca)
```

output:

```python
plt.title("Elbow Method")
plt.xlabel("Number of Clusters (K)")
plt.ylabel("WCSS")
plt.show()
kmeans = KMeans(n_clusters=5, random_state=0)
clusters = kmeans.fit_predict(X_pca)
plt.scatter(X_pca[:, 0], X_pca[:, 1], c=clusters)
plt.title("K-Means Clustering with PCA")
plt.xlabel("PCA Component 1")
plt.ylabel("PCA Component 2")
plt.show()
```

```
C:\Users\Administrator\PycharmProjects\11249a359\11249A359\.venv\Scripts\pyth
Explained Variance Ratio: [0.6538142  0.28839714]
```


Elbow Method
WCSS
500
400
300
200
100
0
2
4
6
8
10
Number of Clusters (K)

K-Means Clustering with PCA
PCA Component 2
2.0
1.5
1.0
0.5
0.0
−0.5
−1.0
−2
−1
0
1
2
PCA Component 1